

Chapter 1: Foundations of KV Caching

Key-Value Caching in Transformers: In transformer-based LLMs, each attention layer computes **key** and **value** vectors from each token, which are then used in dot-product attention. In autoregressive generation, the model generates tokens one by one, and at each step it must attend to all prior tokens. Computing keys/values from scratch for the entire context at every step is wasteful. **KV caching** solves this by storing the keys and values from previous tokens so they can be **reused**. As one blog explains, without caching a 100-token output would force the model to recompute attention on all prior tokens 100 times, whereas caching stores those intermediate results “once and reuse[s] them” ¹. In practice, the cached keys/values (often called *past key-values*) grow with each generated token, enabling each new token’s query to be compared only against previously computed keys/values, not recomputed ones ² ³.

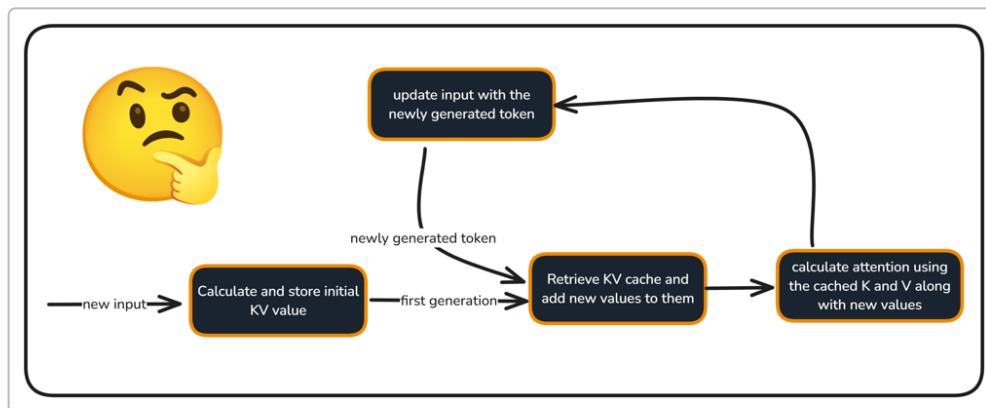


Figure 1. KV-cache workflow in autoregressive generation: initial keys/values are computed and stored; for each new token, the model retrieves and appends new KV entries, and computes attention ($Q \cdot K$) using the cache ².

Autoregressive Generation with KV Cache (Step-by-Step): The cached workflow proceeds roughly as follows (adapted from HuggingFace’s blog) ⁴ : - **First token:** Compute queries, keys, and values. Store the keys and values in the cache. - **Generate token:** Produce the first new token (e.g. “hello”) normally. Append its key/value vectors to the cache. - **Next tokens:** For each subsequent token, **reuse** all cached keys/values from *all previous tokens*. Compute only a new *query* vector for the partial sequence. Run the attention using the stored keys/values plus any new ones. This avoids re-encoding past tokens. - **Update cache:** Append the newly generated token’s keys/values to the cache. Repeat.

This looping means the model never recomputes attention for past tokens from scratch. Instead it does a single QK^T and softmax for the new query against the growing cache each time. The result is a dramatic speedup in the autoregressive decode phase. For example, Hugging Face reports that on a T4 GPU generating the first 300 tokens of a small model took 61.2s **without** caching but only 11.7s **with** KV cache – a $\sim 5.2\times$ speedup ⁵. In other words, KV caching turns a quadratic or repeated workload into a linear one, trading memory (to store the cache) for computation savings.

Cache Structure: In practice, each transformer layer maintains its own cache of keys and values (for each attention head). The cache is typically a tensor of shape *(batch, num_heads, seq_len_so_far, head_dim)* for keys (and similarly for values). As the sequence grows, these tensors grow in length. In multi-head attention each head's keys and values are cached separately. This per-layer, per-head caching is transparent to the user when using high-level APIs, but understanding it explains why cache memory can become large for long sequences or many layers.

Memory vs Speed Trade-off: The key insight is that KV caching **trades memory for speed**. Storing all past keys/values can use significant GPU memory (for large models, longer contexts, and large batches). For example, a 13B-parameter model can require ~800KB *per token* of KV cache ⁶, so a 2048-token context could take ≈1.6 GB of cache. However, this memory overhead is worthwhile in many scenarios: it avoids re-running the expensive attention key/value projections on each step. NVIDIA notes that during decoding the process becomes *memory-bandwidth-bound* (data movement dominates computation), so caching reduces that overhead ⁷. Properly managing this memory (fixed versus dynamic sizes, offloading, etc.) is thus crucial, as we discuss below.

Chapter 2: Framework-Level Implementations

2.1 Hugging Face Transformers

Hugging Face's **Transformers** library provides built-in KV caching support via the `past_key_values` API and a set of cache management classes ³ ⁸. By default, high-level generation methods (`model.generate`) enable caching (`use_cache=True` by default ⁹). The library offers several cache *implementations* for different use cases:

- **DynamicCache (default):** A flexible cache that grows as new tokens arrive. This is the standard mode used when `use_cache=True`. It automatically appends new keys/values to the stored tensors each step. Because its size is not known ahead of time, it does *not* allow certain JIT optimizations. However, it is general and works for any context length.
- **StaticCache:** A fixed-size cache that pre-allocates a maximum length (for all heads and layers) before generation ¹⁰. Static caching enables ahead-of-time compilation and can give up to ~4× speedups in the attention computation, since the maximum tensor sizes are known ⁷ ¹⁰. The trade-off is that many of the pre-allocated positions may be unused (masked) if the actual sequence is shorter, meaning wasted compute. Thus static caches work best when you know an upper bound on context length (e.g. many sequences of similar length). Hugging Face shows how to enable this by `cache_implementation="static"` in `generate()` ¹¹.
- **Offloaded Cache:** For memory-constrained settings, Transformers supports caching where only the current layer's KV is kept on GPU, and the rest are *offloaded* to CPU ¹². During each forward pass, it asynchronously moves the next layer's cache back onto GPU and writes out the completed layer's cache to CPU. This saves GPU memory but with some throughput cost (due to PCIe transfers) ¹² ¹³. One can enable this via `cache_implementation="offloaded"` or by instantiating the cache class with `offloading=True` ¹⁴. This is useful if the model OOMs on large contexts.

- **QuantizedCache:** Transformers vX supports a `QuantizedCache` mode which stores KV tensors in low precision (2, 4, or 8 bits) to reduce memory ¹⁵ ¹⁶. It currently offers two backends: **hqq** (supporting 2-/4-/8-bit) and **quanto** (2-/4-bit) ¹⁵. The example docs recommend tuning parameters (e.g. axis-key/value) for these. Quantized caching trades memory for a bit of quant/dequant overhead; HF notes that if memory is ample, quantizing can actually *hurt* latency ¹⁷. Use `cache_implementation="quantized"` to enable it and pass a config dict (e.g. `backend: "hqq"`).

- **Iterative (Chat) Generation:** In multi-turn chat-like usage, one typically keeps a running `past_key_values` object across turns. Transformers supports starting with an empty `Cache` (e.g. a `DynamicCache` instance), and passing it into successive `generate()` calls. The cache is updated internally. For example, one can do:

```
past = DynamicCache(config=model.config)
for user_input in conversation:
    inputs = tokenizer(user_input, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, past_key_values=past,
                             max_new_tokens=50)
    # model.decode and append to conversation...
```

As Hugging Face docs show, this efficiently retains all KV history without recomputing ¹⁸. (For chat models, one must also handle formatting the prompts, e.g. with `tokenizer.apply_chat_template` ¹⁸.)

- **Prefilled (Prefix) Cache:** You can also precompute a cache for a *fixed prompt* to reuse it across multiple generations. For instance, if you have a standard system instruction, you can run the model once on that prompt to get its KV states (under `with torch.no_grad()`), then copy that cache and reuse it when generating different continuations ¹⁹ ²⁰. This “prefix caching” avoids recomputing shared context. The Transformers docs include an example: they initialize a `StaticCache`, run the model to fill it with the system prompt, then clone that cache for each subsequent query ¹⁹ ²⁰.

Hugging Face also supports **encoder-decoder** models with an `EncoderDecoderCache` that manages both self- and cross-attention caches ²¹, and even model-specific caches for unusual architectures (e.g. Mamba models with no KV states ²²).

2.2 vLLM (PagedAttention)

The **vLLM** engine (developed by UC Berkeley’s Sky Lab) introduces a novel cache management called **PagedAttention** ²³ ²⁴. Instead of preallocating one giant block per request, vLLM breaks each sequence’s KV cache into small fixed-size *pages* (e.g. blocks of 16 tokens per page) ²⁵. It maintains a **virtual→physical** mapping table (like an OS’s page table) so that the model *believes* it has one contiguous context, but the actual data blocks may be scattered in GPU memory ²⁶. The benefits are: (1) **Memory sharing:** If multiple requests share the same prefix, their pages can point to the same physical storage rather than duplicating. (2) **Eliminate waste:** Only pages that are actually used are allocated; no huge all-0

buffers. This approach yields “near-zero waste in KV cache memory” and 2–4× throughput improvement over baseline frameworks ²³ ²⁷. Benchmarks with vLLM show that shared pages (e.g. for a common system prompt) dramatically cut memory usage ²⁷. For example, storing a 3-block system prompt for 3 users only uses 38.4 KB instead of 166.4 KB by sharing ²⁸.

Implementation-wise, vLLM is a full LLM serving system. Users typically write something like:

```
from vllm import LLM
llm = LLM(model="facebook/opt-6.7b", tensor_parallel_size=4,
max_num_batched_tokens=8192)
results = llm.generate(["Prompt 1", "Prompt 2", ...])
```

Internally, vLLM handles packing requests and managing the paged KV caches. It also includes advanced schedulers: e.g. if KV cache memory fills up, vLLM can *preempt* (evict or recompute) requests to free space, and it can **chunk** large prefills to better mix compute-bound (prefill) and memory-bound (decode) tasks ²⁹ ³⁰. In practice, users tune settings like `gpu_memory_utilization` or `tensor_parallel_size` to avoid preemption ³¹ ³². The key point: vLLM’s implementation emphasizes **efficient memory use** via block-based caches and scheduling, at the cost of a more specialized runtime.

2.3 FlashAttention

FlashAttention is a GPU kernel/library (from Dao-AILab) that reorders the attention computation for better efficiency and lower memory movement. In theory, it speeds up the attention phase significantly. NVIDIA notes that “FlashAttention improves performance by minimizing memory movement costs” ³³. In practice, Transformers can plug in FlashAttention for faster inference. However, incremental (autoregressive) decoding introduces a wrinkle. A known issue is that FlashAttention expects the query and key/value tensors to be *aligned* (e.g. right-aligned) in a certain way. In Hugging Face’s implementation, to use FlashAttention during generation one often has to pad the query so that its relative positions match the full key tensor. The FlashAttention GitHub issue explains that without special care, the incremental query would need to be zero-padded (and then masked) to align with the existing keys ³⁴. In short, FlashAttention can **accelerate each attention computation**, but does not by itself reduce the need to cache keys/values. One must still store all keys/values in memory as usual, so FlashAttention’s gains are on computation not cache size. It’s also worth noting that FlashAttention’s default implementation is not compatible with quantized KV caches ³⁵, which can limit some optimizations.

2.4 NVIDIA TensorRT-LLM

NVIDIA’s **TensorRT-LLM** is an end-to-end inference library for LLMs. It includes sophisticated KV cache management features in its engine. In fact, TensorRT-LLM supports multiple cache modes: **paged KV cache** (similar to vLLM’s idea) ³⁶, **circular (ring) KV cache**, **quantized KV cache**, and advanced reuse strategies ³⁶. Two notable new features (as of early 2025) are: (1) **Priority-based cache eviction**, where the user can assign “retention priority” to ranges of tokens. For example, you could mark the system prompt tokens as high priority so they stay longer in cache, while non-critical parts can be evicted sooner ³⁷ ³⁸. NVIDIA reports that this flexible policy can improve cache hit rates by ~20% (vs plain LRU) ³⁸. (2) A **KV cache event**

API, which lets the system observe which cache blocks are used or evicted, enabling higher-level routing/scheduling decisions based on cache state ³⁹ .

TensorRT-LLM also manages memory proactively. Its runtime pre-allocates a pool of KV cache “pages” at initialization and hands them out as requests come in ⁴⁰ . By default it may use, e.g., 90% of free GPU memory for caches, or use user-set `maxTokens` limits ⁴¹ . Internally it can distribute batches as long as enough cached space remains ⁴² . All of this is exposed through its Python API or Triton. In summary, TensorRT-LLM provides a **production-grade, GPU-accelerated stack** where you can tune caching at a fine level: choose paged vs circular cache, enable 8-bit or 16-bit KV, assign cache priorities, etc ³⁶ ³⁸ . The trade-off is complexity: one typically builds a TensorRT engine in advance and then serves requests under its policies.

Comparison Table: The following table summarizes some key differences in how each framework handles KV cache:

Framework	KV Cache Strategy	Throughput Impact	Memory Use	Notes
Hugging Face (HF)	Dynamic/static (on-device) + offload	Baseline (1×) w/ caching ⁵	Moderate	Flexible; easy API use; many modes ³ ¹⁰
vLLM (PagedAttn)	Paged (virtual memory blocks)	2–4× faster on batched loads ⁴³	High efficiency (shared pages)	Special runtime; preemption; continuous batching
FlashAttention	Optimized attention kernel	Speedup in raw attention ops; no KV memory reduction ³⁴	NA (stores full KV)	GPU-opt kernel; requires query padding in incremental mode ³⁴
TensorRT-LLM	Paged/quantized/circular + eviction	Highly optimized (varies)	Very high (quant, shared)	Priority eviction, KV events, engine-based

Chapter 3: Use Cases

3.1 Optimizing Inference Latency and Throughput

KV caching is fundamentally about **speeding up decoding**. By avoiding re-computation, we reduce per-token latency. In batch (high-throughput) settings, caching also helps GPU utilization. NVIDIA notes that during generation the GPU is underutilized (memory-bound) unless requests are batched ⁷ . With caching, one can increase batch size safely since the extra weight is the KV cache, not extra computation. For example, a benchmark showed that using vLLM (with KV optimizations) achieved ~2–4× higher tokens/sec

than the standard Transformers pipeline across various batch sizes ⁴³. In practice, to maximize throughput you often:

- **Batch requests:** Group many prompts together. The KV cache of each request is independent, but batching leverages shared weight data. vLLM and TensorRT both support fully dynamic batching of generations.
- **Use faster hardware kernels:** Employ FlashAttention or fused kernels for the self-attention math (note: these only affect compute, not cache size) ³³.
- **Enable large sequence windows:** If you need very long context, frameworks like vLLM and TensorRT handle it more gracefully (see below). In HF Transformers, you might compile with a static cache to use JIT optimizations, which can boost speed up to 4× ¹⁰ ³³.

For low-latency scenarios (e.g. chatbots), you might prefer *inter-token latency* optimizations. For example, vLLM's chunked-prefill strategy prioritizes generating the next token quickly by interleaving prefills and decodes ⁴⁴. In general, enabling KV caching always reduces *per-token* compute. HF's guide notes that simply turning on `use_cache=True` in `model.generate()` yields large speedups (e.g. 5× in their example) at no accuracy cost ⁵.

3.2 Caching Policies and Dynamic Reuse

In deployment, one often applies strategies around the cache:

- **Offloading / Multi-GPU:** If GPU memory is tight, use cache offloading (HF) or multi-GPU sharding (increase `tensor_parallel` or pipeline parallel in vLLM/RTX). Offloading sacrifices throughput but avoids OOM ¹².
- **Prefetching and asynchronous workflows:** Some systems prefill contexts (like RAG contexts) on CPUs or other nodes. For example, NVIDIA's Triton can be used to do disaggregated prefill, then send only needed cache pages to GPUs.
- **Cache eviction policies:** As mentioned, TensorRT-LLM lets you tailor LRU eviction by token-range priority ³⁷ ³⁸. Custom systems (like LMCache) can also share cache across processes for common prompts.
- **Cache filling reuse:** In multi-turn dialog, one normally *continues* using the same cache object. This means you never throw away history. Transformers' `generate` will accept a `past_key_values` from a previous `generate` call, enabling unlimited context growth (until memory hits) without recomputing history ¹⁸. In fact, the **Attention Sink** technique (see below) leverages this by keeping only a sliding subset of tokens in the cache.
- **Sliding-window models:** Some recent models (Mistral, Gemma, Llama4) use attention that only looks at a fixed window of past tokens. In those cases, the cache naturally caps at the window size. HF docs note that for sliding or chunked attention layers, the cache will never grow beyond the model's window size ⁴⁵ ⁴⁶. This is a built-in policy – older tokens simply fall out of the window and are not attended to.

In summary, practical caching often involves mixing *static pre-caching* for repeated prompts (prefix caching), *dynamic continuation* for dialog, and *memory management* strategies (offload, eviction, quantization) to keep hardware happy.

3.3 Multi-Turn Conversations and Long Contexts

For chatbots or multi-round QA, KV caching is essential. You simply carry forward the cache as each user and assistant turn is appended. This way, each new user prompt is attended against *all* previous turns via the cache. Transformers provide utilities like chat templates, but the core idea is:

```
past_cache = DynamicCache(config=model.config)
for turn in conversation:
    inputs = tokenizer(turn, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, past_key_values=past_cache,
                             max_new_tokens=128)
    # decode and append to conversation; past_cache is updated in place
```

This avoids re-encoding the entire dialogue at each turn ¹⁸. The HF docs example shows two user queries fed sequentially into a `generate` loop using one `past_key_values` ¹⁸. The model “remembers” the previous content via the cache without re-processing it.

Long Context Windows: Long sequences mean large caches. Some strategies: - **Sliding-window/cache trimming:** Models like Mistral use local (sliding) attention by design, so tokens older than `W` just leave the cache. HF notes this explicitly ⁴⁵ ⁴⁶. This bounds memory linearly by the window size, not total history. - **Attention Sinks (SinkCache):** An advanced technique (from the *Attention Sinks* paper) is to treat the first few tokens of the context as “sink tokens” that are always retained in cache, while older non-sink tokens are gradually discarded. Hugging Face even provides a `SinkCache` mode for generation ⁴⁷. Essentially, the model keeps, say, the first 4 tokens (which often include system prompts or key instructions) and forgets others as it generates. This allows continued generation beyond the nominal max length at the cost of losing some earlier context information ⁴⁷. It can be seen as a middle ground between infinite memory and strict windowing. - **Cache Compression:** As context grows, some research suggests selectively dropping low-impact tokens. For example, the *StreamingLLM* approach keeps first tokens plus recent ones, while *Scissorhands* and *H2O* algorithms drop tokens with lowest attention scores to compress the cache ⁴⁸ ⁴⁹. These methods can cut KV size by up to ~80% with little accuracy loss ⁴⁹, but they are not standard in current libraries. They illustrate a general pattern: not all tokens deserve long-term caching, so adaptive eviction can help. - **Memory Cost:** In any case, expect that **KV cache size grows linearly with context length, model size, and batch size** ⁵⁰. This is the fundamental scaling challenge: more tokens means proportionally more key/value data to store. Hence, practitioners often cap context lengths, trim chat histories, or use the above strategies to keep cache size manageable.

Chapter 4: Advanced Topics

4.1 Quantization and Cache Compression

Quantized KV Caches: To reduce memory, some frameworks support storing the cache in lower precision. We discussed HF’s `QuantizedCache` ¹⁵. Similarly, vLLM and TensorRT-LLM allow an 8-bit cache (and newer NVTRT even supports INT8). A recent comparison notes that both vLLM and TRT-LLM typically use **8-bit (FP8)** quantization for KV, because going below that (4- or 2-bit) can severely degrade accuracy ⁵¹.

SqueezeBits reports that quantizing from 16 to 4 or 2 bits can reduce cache size 4–8×, but at a cost to model quality ⁵¹. Therefore, in practice many use 8-bit. For example, TRT-LLM can do FP8 (E4M3) or INT8, and even perform the attention op in FP8 if the whole model is in FP8 mode ⁵². vLLM supports FP8 (E4M3/E5M2) but not INT8 ⁵²; note that its default FlashAttention backend cannot work with quantized KV, so one must switch to another kernel to use quant cache ³⁵. In HF, `QuantizedCache(backend="hq", axis-key=1)` is an example of enabling 4-bit storage ¹⁷.

Multi-Query / Grouped-Query Attention: Another way to save KV memory (less widely available) is to use multi-query or grouped-query attention. In *multi-query* attention, all heads share a single key vector (and value), instead of each having its own. This means the KV cache tensor is smaller by a factor roughly equal to the number of heads. NVIDIA notes that MQA and grouped query attention “reduce memory required by KV caches” ³³. For instance, a model with 16 heads but only 1 set of keys per layer would need 16× less KV storage. This is a model architecture choice, not a runtime trick, but some LLMs (e.g. Mistral-7B) use it partly for efficiency. Using MQA/GQA effectively shifts work from memory to compute, which can be worthwhile if memory is the bottleneck.

Cache Compression Algorithms: Beyond quantization, research has proposed *pruning* the cache. The StreamingLLM paper builds a sliding cache (always keep first tokens + last tokens) ⁵³. The *H2O* and *Scissorhands* methods each drop tokens with low cumulative attention scores to enforce a fixed cache budget ⁵⁴ ⁴⁹. FastGen adaptively selects which tokens to keep based on a target error tolerance ⁵⁵. While promising (up to 80% cache reduction ⁴⁹), these require custom inference logic and are not yet in major libraries. They highlight an ongoing area of innovation: **losing some context to limit memory**.

4.2 Fine-Tuning and Resuming with Precomputed Cache

Fine-Tuning: During training or fine-tuning, KV caching is usually not employed, since training typically processes full contexts in parallel (no autoregressive loop). Most frameworks do not maintain or use a `past_key_values` cache during training mode – doing so would complicate gradient flow and is not standard practice ⁵⁶. We did not find any references to caching being used across gradient updates. In short, KV cache is primarily an *inference* feature.

Checkpoint/Resume: One might ask if it’s possible to save a cache to disk and reload it to resume generation (e.g. saving a 1000-token context so you can later continue from token 1001 without recomputing the first 1000). As of now, mainstream libraries have no built-in checkpoint for cached states. The `past_key_values` can in principle be stored (they are just tensors), but you’d have to manually save and reload them yourself. We found no official support or documentation on “checkpointing” a running decode with a cache. In practice, very long texts are either generated in one session or by recomputing key-values as needed. An exception is the SinkCache method ⁴⁷, which intentionally discards old keys, effectively “resetting” the cache window during decoding.

Memory vs Compute Trade-offs: Throughout, KV caching exemplifies classic trade-offs. A large on-device cache means minimal extra computation but high memory use. A small or offloaded cache saves memory but means some recomputation (or transfer overhead) each token. For example, a *StaticCache* wastes compute (masking many slots) but enables compiled kernels ¹⁰. An *Offloaded* cache saves GPU RAM but incurs PCIe round-trips ¹². Quantizing a cache shrinks it but adds dequant overhead ¹⁷. Model architects can also balance by reducing heads (fewer KV), using local attention (smaller effective `seq_len`), or using

sliding windows (Mistral-style). Best practice is to profile: if GPU RAM is ample, use a full float cache for speed; if memory is limited, employ offloading or 8-bit caching. In clustered serving, sharing or evicting caches based on request similarity can amortize cost ³⁷ ¹⁹ .

Chapter 5: Code Examples and Diagrams

Below are illustrative examples for KV caching usage in PyTorch/Hugging Face Transformers. These are simplified snippets to show key ideas:

• Basic generation with KV cache (HF Transformers):

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2").to("cuda")
prompt = "Once upon a time"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
# By default, use_cache=True, which will store past_key_values
outputs = model.generate(**inputs, max_new_tokens=50)
text = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(text)
```

Here, `model.generate()` runs the autoregressive loop and under the hood caches keys/values for each generated token. The next call to `generate` could even reuse the same model with a `past_key_values` if desired.

• Prefilling a cache for a common prefix (Prefix Caching):

```
from transformers import AutoModelForCausalLM, AutoTokenizer, StaticCache
import torch
import copy

model_id = "gpt2-medium"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id).to("cuda")

# Define a common prompt to cache (e.g. system instruction).
prefix = "You are a helpful assistant. "
# Initialize a static cache big enough for expected prefix length.
cache = StaticCache(config=model.config, max_cache_len=50)
inputs = tokenizer(prefix, return_tensors="pt").to(model.device)
# Prefill the cache with the prefix (no grad needed).
with torch.no_grad():
```

```

cache = model(**inputs, past_key_values=cache).past_key_values

# Now reuse this cache for multiple different continuations:
questions = ["How are you?", "Tell me a joke about cats."]
for q in questions:
    text = prefix + q
    inputs2 = tokenizer(text, return_tensors="pt").to(model.device)
    # Copy the cached prefix keys/values for fresh generation.
    pkv = copy.deepcopy(cache)
    outputs = model.generate(**inputs2, past_key_values=pkv, max_new_tokens=30)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
    print(answer)

```

In this example, we run the model once on `prefix` to get its `past_key_values`, then reuse that cache for each question by copying it ¹⁹ ²⁰. This avoids re-running the common prefix every time.

• Iterative chat with DynamicCache:

```

from transformers import AutoTokenizer, AutoModelForCausalLM, DynamicCache
import torch

model_id = "meta-llama/Llama-2-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id).to("cuda")
conversation = [
    {"role": "user", "content": "Hello, who are you?"},
    {"role": "assistant", "content": "I am an AI assistant."}
]

past = DynamicCache(config=model.config)
for turn in conversation:
    text = tokenizer.apply_chat_template([turn], add_generation_prompt=True)
    inputs = tokenizer(text, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=50, past_key_values=past)
    reply = tokenizer.decode(outputs[0, -50:], skip_special_tokens=True)
    print(reply)

```

Here we manually iterate through turns, reusing the `past` cache each time ¹⁸. The `DynamicCache` accumulates all KV states, so each new reply attends to the full dialogue history.

Chapter 6: Research Landscape and Best Practices

Academic Papers & Benchmarks: The idea of KV cache and its optimization is relatively recent in research. The PagedAttention mechanism (vLLM) was published at SOSP 2023 ²³, demonstrating the core memory- and throughput benefits (2–4× speedups, near-zero waste). Other works (e.g. *Attention Sinks* (arXiv 2023)

underlying HF's SinkCache, *StreamingLLM*/H2O/Scissorhands/FastGen) explore compressing caches based on token importance ⁴⁸ ⁴⁹ ⁴⁷. Neptune.ai and other blogs have analyzed KV cache sizes and practical tips. The NVIDIA inference blog summarized key optimizations (multi-query, FlashAttention, PagedAttention) at a high level ³³ ⁵⁷. GitHub issues (e.g. FlashAttention incompatibilities) and benchmarks (like the "vLLM vs HF" repo ⁴³) provide real-world comparisons. In general, sources concur that **KV caching is a critical leverage point**: it is one of the biggest bottlenecks in inference, and getting it right yields large wins.

Best Practices: Across sources, the advice includes: - *Always enable caching* (`use_cache=True`) for generation unless you specifically need full forward passes (some sampling algorithms might). - *Profile memory usage*: know if your workload is memory- or compute-bound. If memory is the limit, consider static compilation, offloading, or quantization ¹⁰ ¹⁷. If compute is the limit (e.g. long beam search), focus on faster kernels (FlashAttention) and batching. - *Tailor to model*: Large context models (Gemma, Llama4) already cap cache via their attention; for others, use sliding or sink caches if context is truly huge. - *Tune framework settings*: vLLM recommends raising `max_num_batched_tokens` for better throughput ⁵⁸, or adjusting `gpu_memory_utilization` to avoid preemption ³². HuggingFace suggests enabling `torch.compile` when using static cache for maximum speed ¹¹. - *Reuse and share*: In production, share cache across requests when possible (e.g., multiple variations of a query with same prefix). Use priority policies (as in TensorRT-LLM) to hold important KV in cache longer ³⁷ ³⁸. - *Monitor and adapt*: Watch for slow-downs or OOMs. Modern libraries expose metrics (e.g. vLLM's preemption logs or TensorRT memory usage) so you can adjust configuration.

Ongoing Challenges: Despite advances, long-context inference still pushes memory limits. The community is actively researching cache compression (to make GPTs with 100k+ context tokens feasible) and better attention kernels. There's also interest in "KV cache offloading" across devices or even to high-bandwidth memory (HBM) to serve extremely large models. Another challenge is multi-modal or multi-query generation, which complicates caching.

In summary, KV caching is now a **core technique** in LLM inference. Mastering it involves understanding both the high-level (framework APIs, generation loops) and the low-level (tensor memory, hardware). By combining the methods above — efficient frameworks (vLLM, TensorRT), smart caching strategies (static vs dynamic, offload, quantization), and pragmatic system tuning — practitioners can significantly boost inference throughput. The field is rapidly evolving, but the principles remain: cache what you can, and reuse it wherever possible.

Sources: Discussions and documentation from Hugging Face ³ ¹⁰ ¹⁵ ¹⁸, vLLM documentation and blog ²⁴ ⁴³ ²⁹, NVIDIA's developer blogs ³³ ³⁶ ³⁸, and academic/blog analyses ⁴⁹ ⁴⁷ ⁵² ²³ were used to compile this guide. These collectively cover the current state-of-the-art in KV caching.

¹ ⁶ ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ The Architecture Behind vLLM: How PagedAttention Improves Memory Utilization
| by Mandeep Singh | Medium

<https://medium.com/@mandeep0405/the-architecture-behind-vllm-how-pagedattention-improves-memory-utilization-2f9b25272110>

² ⁴ ⁵ ⁹ KV Caching Explained: Optimizing Transformer Inference Efficiency

<https://huggingface.co/blog/not-lain/kv-caching>

3 8 10 11 12 13 14 15 16 17 18 19 20 21 22 45 46 KV cache strategies

https://huggingface.co/docs/transformers/en/kv_cache

7 33 57 Mastering LLM Techniques: Inference Optimization | NVIDIA Technical Blog

<https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>

23 [2309.06180] Efficient Memory Management for Large Language Model Serving with PagedAttention

<https://arxiv.org/abs/2309.06180>

29 30 32 44 Performance and Tuning — vLLM

<https://docs.vllm.ai/en/v0.6.0/models/performance.html>

31 58 Optimization and Tuning - vLLM

<https://docs.vllm.ai/en/stable/configuration/optimization/>

34 Flash attention cannot be fully leveraged with use_cache during inference · Issue #466 · Dao-AILab/flash-attention · GitHub

<https://github.com/Dao-AILab/flash-attention/issues/466>

35 51 52 [vLLM vs TensorRT-LLM] #8. KV Cache Quantization - SqueezeBits

<https://blog.squeezebits.com/vllm-vs-tensorrtllm-8-kv-cache-quantization-35079>

36 37 38 39 50 Introducing New KV Cache Reuse Optimizations in NVIDIA TensorRT-LLM | NVIDIA Technical Blog

<https://developer.nvidia.com/blog/introducing-new-kv-cache-reuse-optimizations-in-nvidia-tensorrt-llm/>

40 41 42 Memory Usage of TensorRT-LLM — TensorRT-LLM

<https://nvidia.github.io/TensorRT-LLM/reference/memory.html>

43 GitHub - alishafique3/vLLM-vs-Hugging-Face: This project benchmarks vLLM and Hugging Face Transformers for offline LLM inference, leveraging vLLM's optimized execution such as PagedAttention and continuous batching, to enable faster generation and efficient GPU memory usage.

<https://github.com/alishafique3/vLLM-vs-Hugging-Face>

47 transformers-community/sink_cache · Hugging Face

https://huggingface.co/transformers-community/sink_cache

48 49 53 54 55 LLM Inference Series: 4. KV caching, a deeper look | by Pierre Lienhart | Medium

<https://medium.com/@plienhar/llm-inference-series-4-kv-caching-a-deeper-look-4ba9a77746c8>

56 transformers/src/transformers/cache_utils.py at main · huggingface/transformers · GitHub

https://github.com/huggingface/transformers/blob/main/src/transformers/cache_utils.py